# 1 Introduction

## 1.1 OPC Foundation

The use of PC- and software-based automation systems in industrial automation rapidly increased since the early nineties. Especially, Windows-based PCs are used for visualization and control purposes. One of the major efforts for the development of standardized automation software in the past years was the access to automation data in devices where an uncountable number of different bus systems, protocols, and interfaces are used.

A similar problem for software applications did exist for the access to printers, where in old DOS days, every application needed to write its own printer drivers for all supported printers. Windows solved the printer driver problem by incorporating printer support into the operating system. This one printer driver interface served all applications that needed printer access. And these printer drivers are provided by the printer manufacturer and not by the application developers.

Since vendors of Human Machine Interface (HMI) and Supervisory Control and Data Acquisition (SCADA) software had similar problems, a task force initiated by the companies Fisher-Rosemount, Rockwell Software, Opto 22, Intellution, and Intuitive Technology was founded in 1995. The goal of the task force was to define a Plug&Play standard for device drivers providing a standardized access to automation data on Windows-based systems.

The result was the OPC Data Access specification released after short time in August 1996. The nonprofit organization that is maintaining this standard is the OPC Foundation. Nearly all vendors providing systems for industrial automation became member of the OPC Foundation. The OPC Foundation was able to define and adopt praxis relevant standards much quicker than other organizations. One of the reasons for this success was the reduction to main features and the restriction to the definition of APIs using the Microsoft Windows technologies Component Object Model (COM) and Distributed COM (DCOM). The focus on important features and the use of base Windows technologies allowed a quick adoption of the standard for the addressed use case.

As a result of the experience from product developments, multi-vendor demonstrations, and interoperability workshops, version two of the OPC Data Access specification was introduced in 1998. Based on this version, a large number of products implemented the standard. OPC Data Access version two is still the most important interface for OPC products.

SCADA and HMI systems, process management and Distributed Control Systems (DCS), PC-based control systems, and Manufacturing Execution Systems (MES) must support OPC interfaces today. OPC is the one – universally accepted – standard delivering the ability to exchange data between different industrial automation system in manufacturing and process industry.

W. Mahnke et al., *OPC Unified Architecture*,
DOI: 10.1007/978-3-540-68899-0_1, 

After 12 years, the OPC Foundation has over 450 members including all relevant automation system suppliers around the world. Figure 1.1 shows the OPC Foundation member demography classified with membership classes and region. The membership class is based on sales number for corporate members and the classes for end user and non-voting members like universities or other organizations. The OPC Foundation is governed by a Board of Directors elected by the membership. The Board, in turn, appoints the Foundation's Officers and the OPC Chief Architect. A Marketing Committee, a Technical Advisory Council, and various working groups have been established.

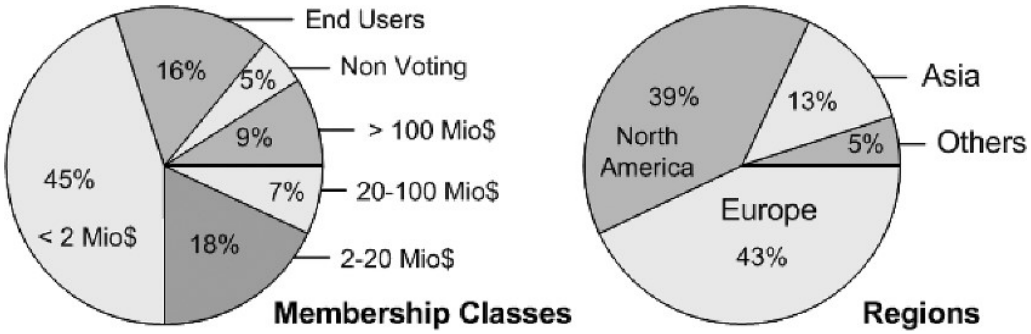


**Fig. 1.1** OPC Foundation member demography

The OPC Foundation has listed over 1,500 OPC-based products in its product catalog containing only products from OPC members. The total OPC market has over 2,500 vendors providing over 15,000 OPC-enabled products.

This great success requires verification mechanisms to make sure that all OPC products interoperate with each other and to ensure a certain level of quality. For this reason the OPC Compliance Program is, beside the development of new standards, the main focus of the OPC Foundation working groups.

The OPC Compliance Program defines two certification levels. The first level combines self certification and interoperability workshops. The OPC Foundation offers Compliance Test Tools for all relevant OPC standards. These tools are used for testing and the encrypted results are sent to the OPC Foundation. These test tools cover the functional tests on the interface level. Interoperability workshops are yearly events in Europe, North America, and Japan, where different vendors can test the interoperability of their OPC products against each other. Products passing self-certification can use the Self-Tested logo to indicate a basic level of OPC Compliance.

The second level is the product certification in independent Certification Test Labs. Accredited third party test labs are verifying OPC products with broader test coverage. In addition to the basic functional tests executed by the Compliance Test Tools, the test labs are running behavior tests, load and stress tests, interoperability tests as well as environment and usability tests. For products passing third party certification, the OPC Certified logo indicates a high level of quality and OPC Compliance.

End users are encouraged to buy only OPC Compliance tested products to reduce interoperability problems and to ensure reliability and performance of their OPC-based solution.

## 1.2 Classic OPC

In recent years, the OPC Foundation has defined a number of software interfaces to standardize the information flow from the process level to the management level. The main use cases are interfaces for industrial automation applications like HMIs and SCADA systems to consume current data from devices and to provide current and historical data and events for management applications.

According to the different requirements within industrial applications, three major OPC specifications have been developed: Data Access (DA), Alarm & Events (A&E), and Historical Data Access (HDA). Access to current process data is described in the DA specification, A&E describes an interface for event-based information, including acknowledgement of process alarms, and HDA describes functions to access archived data. All interfaces offer a way to navigate through the address space and to provide information about the available data.

OPC uses a client–server approach for the information exchange. An OPC server encapsulates the source of process information like a device and makes the information available via its interface. An OPC client connects to the OPC server and can access and consume the offered data. Applications consuming and providing data can be both client and server. Figure 1.2 shows a typical use case of OPC clients and servers.

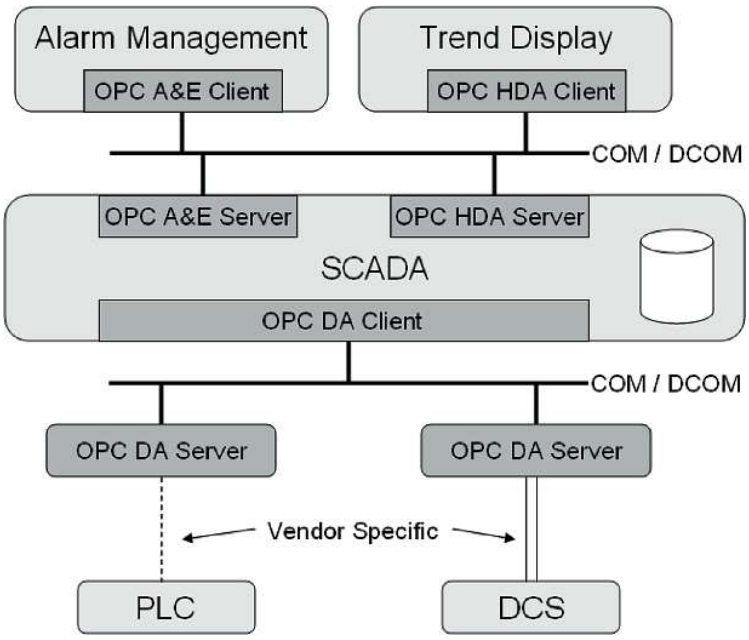


**Fig. 1.2** Typical use case of OPC clients and servers

Classic OPC interfaces are based on the COM and DCOM technology from Microsoft.

The advantage of this approach was the reduction of the specification work to the definition of different APIs for different specialized needs without the requirement to define a network protocol or a mechanism for interprocess communication. COM and DCOM provide a transparent mechanism for a client to call methods on a COM-object in a server running in the same process, in another process, or on another network node. Using this technology available on all PC-based Windows operating systems reduced the development time of the specifications and products and the time-to-market for OPC. This advantage was important for the success of OPC.

The two main disadvantages are the Windows-platform-dependency of OPC and the DCOM issues when using remote communication with OPC. DCOM is difficult to configure, has very long and non-configurable timeouts, and cannot be used for internet communication.

### *1.2.1 OPC Data Access*

The OPC Data Access interface enables reading, writing, and monitoring of variables containing current process data. The main use case is to move real-time data from PLCs, DCSs, and other control devices to HMIs and other display clients. OPC DA is the most important OPC interface. It is implemented in 99% of the products using OPC technology today. Other OPC interfaces are mostly implemented in addition to DA.

OPC DA clients explicitly select the variables (OPC items) they want to read, write, or monitor in the server. The OPC client establishes a connection to the server by creating an OPCServer object. The server object offers methods to navigate through the address space hierarchy to find items and their properties like data type and access rights.

For accessing the data, the client groups the OPC items with identical settings such as update time in an OPCGroup object. Figure 1.3 shows the different objects the OPC client creates in the server.

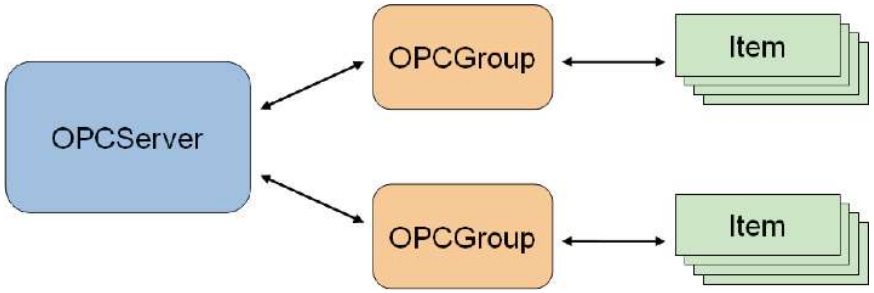


**Fig. 1.3** Objects created by an OPC client to access data

When added to a group, items can be read or written by the client. However, the preferred way for the cyclic reading of data by the client is monitoring the value changes in the server. The client defines an update rate on the group containing the items of interest. The update rate is used in the server to cyclic check the values for changes. After each cycle, the server sends only the changed values to the client.

OPC provides real-time data that may not permanently be accessible, for example, when the communication to a device gets temporarily interrupted. The Classic OPC technology handles this issue by providing timestamp and quality for the delivered data. The quality specifies if the data is accurate (good), not available (bad), or unknown (uncertain).

### *1.2.2 OPC Alarm & Events*

The OPC A&E interface enables the reception of event notifications and alarm notifications. Events are single notifications informing the client about the occurrence of an event. Alarms are notifications that inform the client about the change of a condition in the process. Such a condition can be the level of a tank. In this example, a condition change can occur when a maximum level is exceeded or is fallen below a minimum level. Many alarms include the requirement that the alarm has to be acknowledged. This acknowledgement is also possible via the OPC A&E interface.

OPC A&E thus provides a flexible interface for transmitting process alarms and events from different event sources.

To receive notifications, the OPC A&E client connects to the server, subscribes for notifications, and than receives all notifications triggered in the server. To limit the number of notifications, the OPC client can specify certain filter criteria.

The OPC client connects by creating an OPCEventServer object in the A&E server in the first step and by generating an OPCEventSubscription used to receive the event messages in the second step. Filters for these event messages can be configured separately for each subscription. Figure 1.4 shows the different objects the OPC client creates in the server.

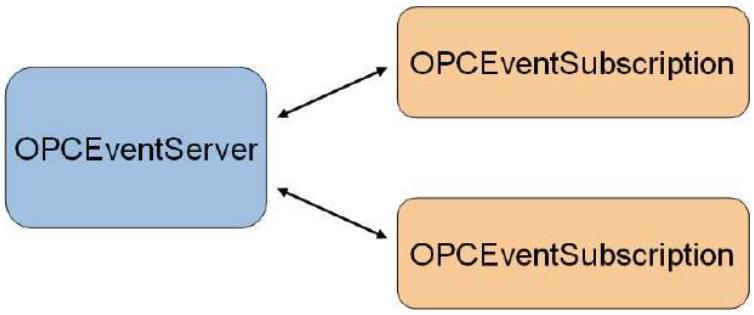


**Fig. 1.4** Objects created by an OPC client to receive events

In contrast to OPC DA, there is no explicit request for specific information like reading values; however, all process events are supplied and the client can limit the quantity of the events by setting certain filter criteria, for example, filter by event types, by priority, or by event source.

### *1.2.3 OPC Historical Data Access*

Where OPC Data Access gives access to real-time, continually changing data, OPC Historical Data Access provides access to data already stored. From a simple serial data logging system to a complex SCADA system, historical archives can be retrieved in a uniform manner.

The OPC client connects by creating an OPCHDAServer object in the HDA server. This object offers all interfaces and methods to read and update historical data. A second object OPCHDABrowser is defined for browsing the address space of the HDA server.

The main functionality is the reading of historical data in three different ways. The first mechanism reads raw data from the archive, where the client defines one or more variables and the time domain he wants to read. The server returns all values archived for the specified time range up to the maximum number of values defined by the client. The second mechanism reads values of one or more variables for specified timestamps. The third read mechanism computes aggregate values from data in the history database for the specified time domain for one or more variables. Values include always the associated quality and timestamp.

In addition to the read methods, OPC HAD also defines methods for inserting, replacing, and deleting data in the history database.

### *1.2.4 Other OPC Interface Standards*

OPC specified several additional standards as base specifications or for specialized needs. Base specifications are OPC Overview and OPC Common defining interfaces and behavior that is common to all COM-based OPC specifications. Figure 1.5 gives an overview for all Classic OPC specifications.

OPC Security specifies how to control client access to servers to protect sensitive information and to guard against unauthorized modification of process parameters.

OPC Complex Data, OPC Batch, and OPC Data eXchange (DX) are extensions to OPC DA. Complex Data defines how to describe and transport values with complex structured data types. OPC DX specifies the data exchange between Data Access servers by defining the client behavior and the configuration interfaces for the client inside a server. OPC Batch extends DA for the specialized needs of batch processes. It provides interfaces for the exchange of equipment capabilities

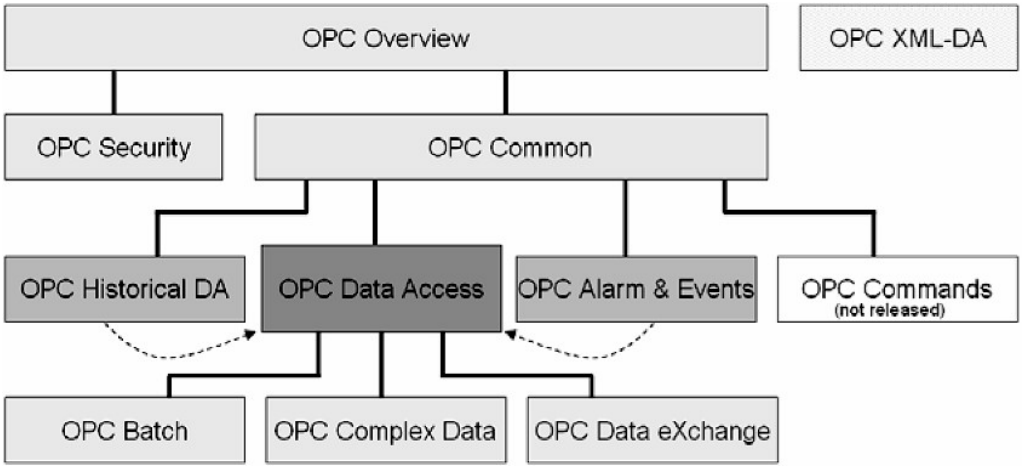


**Fig. 1.5** Classic OPC interface standards

corresponding to the S88.01 Physical Model [ISA88] and current operating conditions.

OPC Commands defines mechanisms to call methods or to execute programs via OPC. This specification was never released since it was finished after OPC UA was started. But its content and functionality is completely incorporated into UA.

### *1.2.5 OPC XML-DA*

OPC XML-DA was the first platform-independent OPC specification replacing COM/DCOM with HTTP/SOAP and Web Service technologies. Thus a vendor- and platform-neutral communication infrastructure was introduced and widely accepted functionality of OPC Data Access was retained.

Since typical Web Services are stateless, the functionality was reduced to the minimum set of methods to exchange OPC Data Access information, without the need for methods to create and modify a context for communication. Only eight methods were needed to cover the key features of OPC Data Access.

The eight services are the following:

- GetStatus to verify the server status
- Read to read one or more item values
- Write to write one or more item values
- Browse and GetProperties to get information about the available items
- Subscribe to create a subscription for a list of items
- SubscriptionPolledRefresh for the exchange of changed values of a subscription
- SubscriptionCancel to delete the subscription.

OPC XML-DA was designed for internet access and enterprise integration. But based on its platform-independence, it was mainly implemented in embedded systems and on non-Microsoft platforms. But due to its high resource consumption

and limited performance, it was not as successful as expected for this type of applications.

## 1.3 Motivation for OPC UA

The first and still most successful Classic OPC standard – OPC Data Access – was designed as interface to communication drivers, allowing a standardized read and write access to current data in automation devices. The major use case was HMI and SCADA systems accessing data from different types of automation hardware and devices from different vendors using one defined software interface supplied by the hardware vendor. Standards following later like OPC Alarm & Events and OPC Historical Data Access were also designed to access information provided by SCADA systems.

With the successful adoption of OPC in thousands of products, OPC is used today as standardized interface between automation systems in different levels of the automation pyramid. It is even used in a lot of areas where it was not designed for, and there are many more areas where manufacturers want to use a standard like OPC but are not able to use it because of the COM dependency of OPC or because of the limitations for remote access using DCOM.

OPC XML-DA was the first approach of the OPC Foundation to maintain successful features of OPC but to use a vendor and platform neutral communication infrastructure. There are several reasons why just creating Web Service versions of the successful OPC specification did not cover the requirements for a new OPC generation. One reason was the poor performance of XML Web Service compared with original COM version. Furthermore, using different XML Web Service stacks caused interoperability problems.

But besides the issue of platform independence, the OPC member companies brought forward the requirement to expose complex data and complex systems, removing the limitations of Classic OPC.

The OPC Unified Architecture was born out of the desire to create a true replacement for all existing COM-based specifications without losing any features or performance. Additionally it must cover all requirements for platform-independent system interfaces with rich and extensible modeling capabilities being able to describe also complex systems. The wide range of applications where OPC is used requires scalability from embedded systems across SCADA and DCS up to MES and ERP systems. The most important requirements for OPC UA are listed in Table 1.1.

The requirements can be grouped into the ones for the communication between distributed systems being able to exchange information and the requirements for modeling of data describing a system and the available information.

Classic OPC was designed as device driver interface. OPC is used as system interface today; therefore, the reliability for the communication between distributed systems is very important. Since network communication is not reliable by

definition, robustness and fault-tolerance are the important requirement, including redundancy for high availability. Platform-independence and scalability is necessary to be able to integrate OPC interfaces directly into the systems running on many different platforms. To replace proprietary communication, an important requirement is always high-performance in intranet environments. But also internet communication through firewalls must be possible out of the box, which makes security and access control another important requirement. And first and foremost the interoperability between systems from different vendors is still the most important requirement.

**Table 1.1** Requirements for OPC UA

| Communication between distributed systems | Modeling Data |
|---|---|
| • Reliability by<br>  • Robustness and fault tolerance<br>  • Redundancy<br>• Platform-independence<br>• Scalability<br>• High performance<br>• Internet and firewalls<br>• Security and access control<br>• Interoperability | • Common model for all OPC data<br>• Object-oriented<br>• Extensible type system<br>• Meta information<br>• Complex data and methods<br>• Scalability from simple to complex models<br>• Abstract base model<br>• Base for other standard data models |

Modeling of data was very limited in Classic OPC and needed to be enhanced by providing a common, object-oriented model for all OPC data. This model must include an extensible type system to be able to offer meta information and to describe also complex systems. The availability of methods provided and described by servers and callable by clients is a powerful feature needed to make OPC flexible and extensible. Complex data is required to support the description and consistent transport of complex data structures. It was an important requirement to enhance the modeling capabilities, but it was equally important to support simple models with simple concepts. For this reason it is required to have a simple and abstract but extensible base model to be able to scale from simple to complex models.

In addition to the functional requirements for a new OPC generation, the initial group of over 40 representatives defining the requirements and use cases for OPC Unified Architecture was not only composed of OPC members. Other standardization organizations like IEC and ISA interested in using OPC as transport mechanism for their information were involved in the early design process. In this group the OPC Foundation defines HOW to describe and transport data, and the collaborating organizations define WHAT data they want to describe and transport depending on their information model.

Another important design goal was to allow an easy migration to OPC Unified Architecture to protect the investment in the very successful Classic OPC standards and to build upon the large installed base of OPC.

## 1.4 OPC UA Overview

To reach the defined goals, the OPC Unified Architecture builds on different layers shown in Fig. 1.6.

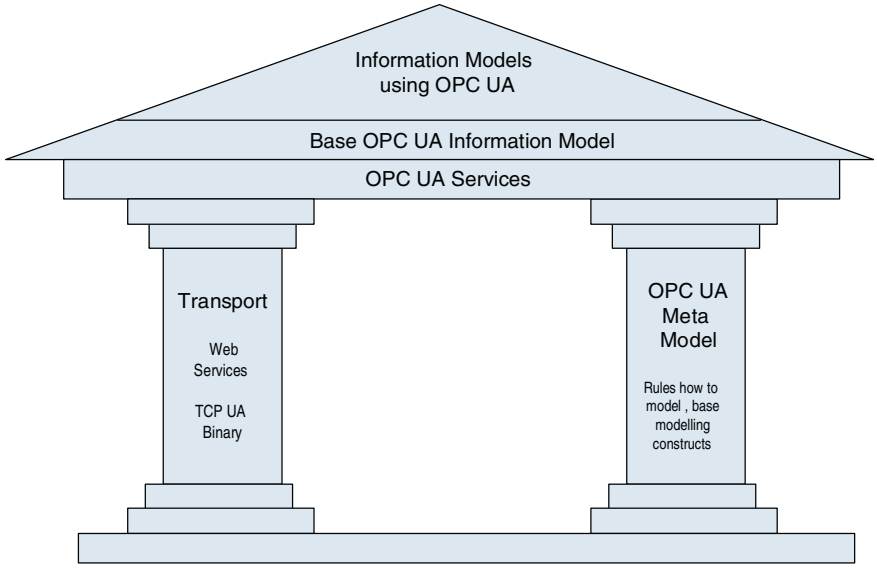


**Fig. 1.6** The foundation of OPC UA

The fundamental components of OPC Unified Architecture are transport mechanisms and data modeling.

The transport defines different mechanisms optimized for different use cases. The first version of OPC UA is defining an optimized binary TCP protocol for high performance intranet communication as well as a mapping to accepted internet standards like Web Services, XML, and HTTP for firewall-friendly internet communication. Both transports are using the same message-based security model known from Web Services. The abstract communication model does not depend on a specific protocol mapping and allows adding new protocols in the future. The transport mechanisms are described more detailed in Chap. 6.

The data modeling defines the rules and base building blocks necessary to expose an information model with OPC UA. It defines also the entry points into the address space and base types used to build a type hierarchy. This base can be extended by information models building on top of the abstract modeling concepts. In addition, it defines some enhanced concepts like describing state machines used in different information models. The basics of information modeling are described in Chap. 2 and an example and best practices are introduced in Chap. 3.

The UA Services are the interface between servers as supplier of an information model and clients as consumers of that information model. The Services are defined in an abstract manner. They are using the transport mechanisms to exchange the data between client and server.

This basic concept of OPC UA enables an OPC UA client to access the smallest pieces of data without the need to understand the whole model exposed by complex systems. OPC UA clients also understanding specific models can use more

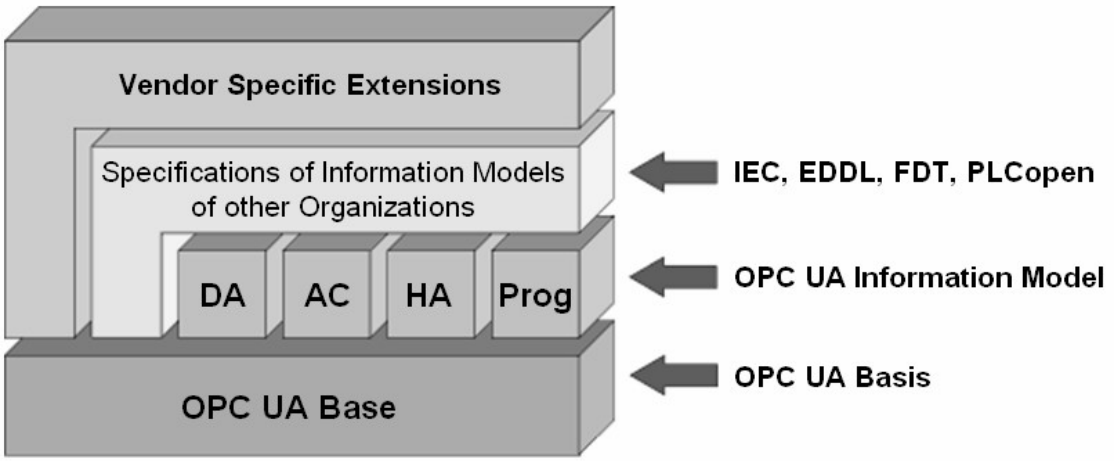


**Fig. 1.7** OPC UA layered architecture

enhanced features defined for special domains and use cases. Figure 1.7 shows the different layers of information models defined by OPC, by other organizations, or by vendors.

To cover all successful features known from Classic OPC, information models for the domain of process information are defined by OPC UA on top of the base specifications. DA defines automation-data-specific extensions such as the modeling of analog or discrete data and how to expose quality of service. All other DA features are already covered by the base. Alarm & Conditions (AC) specifies an advanced model for process alarm management and condition monitoring. Historical Access (HA) defines the mechanisms to access historical data and historical events. Programs (Prog) specifies a mechanism to start, manipulate, and monitor the execution of programs.

Other organizations can built their models on top of the UA base or on top of the OPC information model, exposing their specific information via OPC UA. Examples for standards already working on mappings to OPC UA are Field Device Integration (FDI) combining Electronic Device Description Language (EDDL), and Field Device Tool (FDT) both used to describe, to configure, and to monitor devices and PLCopen, a standard for PLC programming languages.

Additional vendor-specific information models will be defined using directly the UA base, the OPC models, or other OPC-UA-based information models.

## 1.5 OPC UA Specifications

The OPC UA specifications are partitioned in different parts also required for IEC standardization. OPC UA will be known as IEC 62541 standards. Figure 1.8 shows an overview of all specification parts split into the core specifications defining the base for OPC UA and the access type specific parts mainly specifying the OPC UA information models.

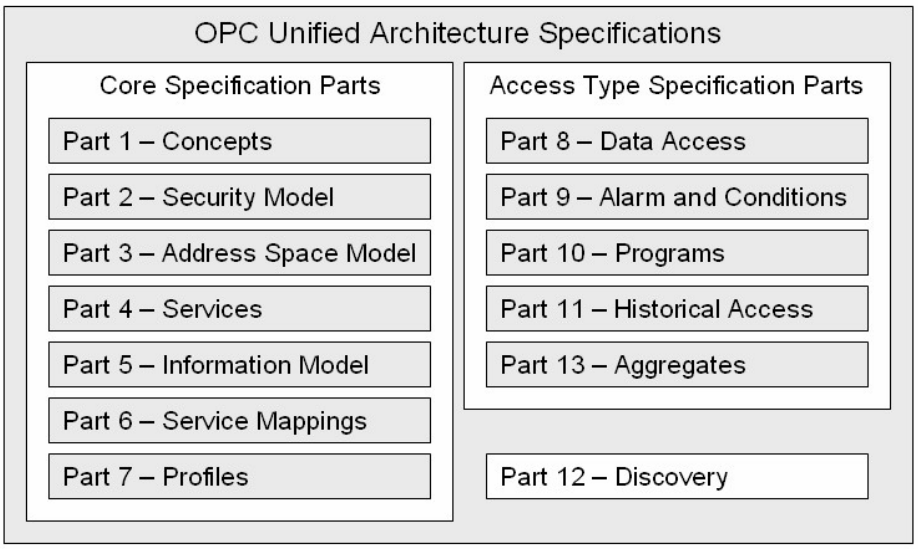


**Fig. 1.8** OPC UA specifications

The first two parts are not normative. The concepts part [UA Part 1] gives an overview about OPC UA and [UA Part 2] describes the security requirements and the security model for OPC UA.

Most important to understand how to model and access information are part 3 and 4. These two specifications are the key documents for the design and development of OPC UA applications.

The Address Space Model [UA Part 3] specifies the building blocks to expose instance and type information and thus the OPC UA meta model used to describe and expose information models and to build an OPC UA server address space.

The abstract UA Services defined in [UA Part 4] represent the possible interactions between UA client and UA server applications. The client uses the Services to find and access information provided by the server. The Services are abstract because they are defining the information to be exchanged between UA applications but not the concrete representation on the wire and also not the concrete representation in an API used by the applications. Figure 1.9 shows the layered communication architecture of OPC UA.

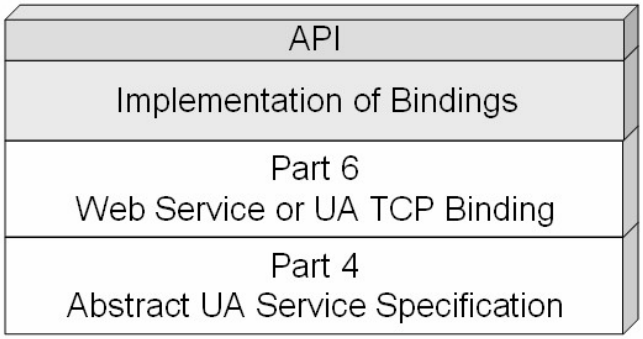


**Fig. 1.9** Layered OPC UA communication architecture

The mapping of the UA Services to messages, the security mechanisms applied to the messages, and the concrete wire transport of the messages are defined in [UA Part 6]. Only implementers of UA stacks need to completely understand this specification. Since the OPC Foundation supplies proper UA stacks, typical UA application architects and programmers do not need to read this specification.

The base information model specified in [UA Part 5] provides the framework for all information models using OPC UA. It defines the following:

- The entry points into the address space used by clients to navigate through the instances and types of an OPC UA server
- The base types building the root for the different type hierarchies
- The built-in but extensible types like object types and data types
- The Server Object providing capability and diagnostic information.

The profiles are defining useful subsets of OPC UA features in [UA Part 7]. Such a subset must be implemented completely by an UA application to ensure interoperability for the defined subset. The specification defines the subsets on two levels. The first level are Conformance Units defining a small set of functionality that is always used together and can be tested with Compliance Test Tools and verified as unit. The second level are Profiles composed of a list of conformance units. A profile must be implemented completely and will be verified as complete set during the certification of OPC UA products. The list of supported and used Profiles is exchanged during the connection establishment between client and server and allows the applications to determine if the needed features are supported by the communication partner.

The DA information model defines how to represent and use automation data and specific characteristics like engineering units in [UA Part 8].

The AC information model specifies process alarm and condition monitoring specific state machines and types of events in [UA Part 9].

The Programs information model defines a base state machine for the execution, manipulation, and monitoring of programs in [UA Part 10].

The HA information model in [UA Part 11] specifies the use of the history access Services and how to present information about the configuration of data and event history.

The aggregates used to compute aggregated values from raw data samples are specified in [UA Part 13]. The aggregates are used for historical access as well as the monitoring of current values.

[UA Part 12] defines how to find servers in the network and how a client can get the necessary information to be able to establish a connection to a certain server.

## 1.6 OPC UA Software Layers

OPC UA uses a similar client–server concept like Classic OPC. An application that wants to expose its own information to other applications is called UA server

and an application that wants to consume information from other applications is called UA client. But it is expected that much more applications will be both UA server and UA client in one application than in Classic OPC. One reason is that more UA servers will be integrated directly in devices. Implementing also a UA client enables device to device communication. Another reason is the use of OPC UA as configuration interface, where UA clients are also UA servers to be configured via OPC UA.

A typical OPC UA application is composed of three software layers shown in Fig. 1.10. The complete software stack can be implemented with C/C++, .NET, or JAVA. OPC UA is not limited to these programming languages and development platforms, but only these environments are currently used for implementing the OPC Foundation UA Stack deliverables.

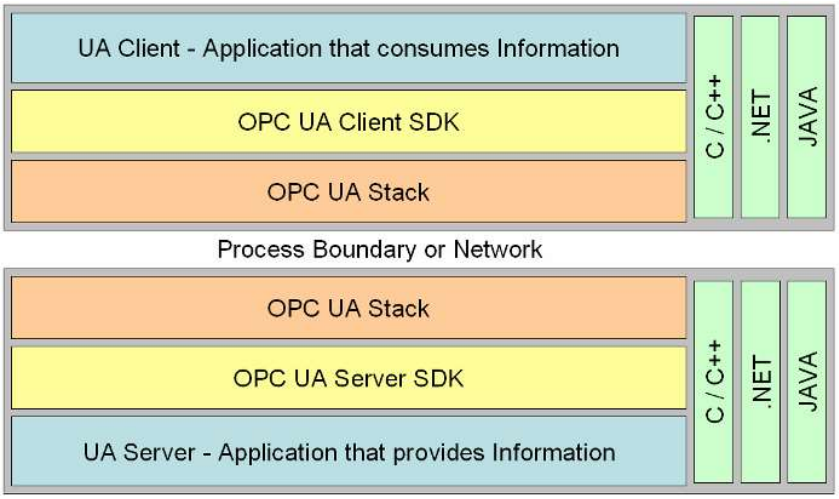


**Fig. 1.10** OPC UA software layers

An OPC UA Application is a system that wants to expose or to consume data via OPC UA. It contains the specific functionality for the application and the mapping of this functionality to OPC UA by using an OPC UA Stack and an OPC UA Software Development Kit (SDK).

An OPC UA client or server SDK implements common OPC UA functionality that is part of the application layer, since the UA Stacks implement only the communication channels. An OPC UA SDK reduces the development effort and facilitates faster interoperability for an OPC UA application.

An OPC UA Stack implements the different OPC UA transport mappings defined in [UA Part 6]. The Stack is used to invoke UA Services across process or network boundaries. OPC UA defines three Stack layers and different profiles for each layer. The message encoding layer defines the serialization of Service parameters in a binary and a XML format. The message security layer specifies how the messages must be secured by using the Web Service security standards or a UA binary version of the Web Service standards. The message transport layer defines the used network protocol, which could be UA TCP or HTTP and SOAP for Web Services. Figure 1.11 illustrates the different UA communication stack layers. The implementation of the layers in a UA Stack and the resulting API for

the applications is not part of the OPC UA specification. The UA Stacks provide language-dependent APIs for UA client and UA server applications, but the Services and their parameters are similar and based on the abstract Service definition in [UA Part 4].

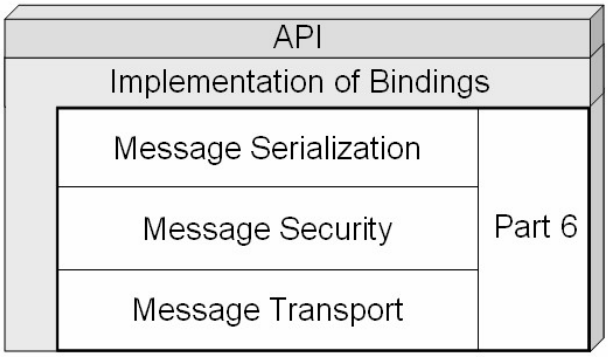


**Fig. 1.11** UA communication stack layers defined in UA Part 6

With implementations in ANSI C/C++, .NET, and JAVA, the main development environments and programming languages are covered by UA Stacks developed and maintained by the OPC Foundation.

## 1.7 Evolution Not Revolution

OPC UA is much more flexible and has much more features than all Classic OPC specifications together, but it incorporates all successful concepts of existing OPC specification, fixes known issues in existing standards, and adds standardization for a lot of additional use cases.

It was an important design goal to allow an easy migration from Classic OPC to OPC UA. For this reason most features known from Classic OPC can be found in OPC UA using sometimes slightly different terminology. It is not possible to expose all UA features with Classic OPC interfaces, but it is no problem to map Classic OPC features to OPC UA.

OPC UA allows a simple mapping and offers migration strategies to integrate OPC products based on previous OPC standards. One part of the migration strategy does not even require a change in existing products. Wrappers and proxies provided by the OPC Foundation are able to translate the different Classic OPC interfaces into OPC UA and vice-versa. This first level in the migration strategy can be used by vendors to support OPC UA for legacy products.

The second level of migration to OPC UA is the integration of OPC UA directly into existing products without adding OPC UA specific features. This step does not require changes in interfaces used between systems and their OPC communication components today. It is much easier to integrate new components if the existing interfaces of a system do not have to be changed. The advantage over the use of wrappers or proxies is a better performance and less configuration and

engineering efforts by removing an additional software layer. The direct integration will make it easier to remove potential limitations of wrappers and proxies and allows an iterative development approach by adding OPC UA features step by step.

The third level may require changing internal interfaces of the product to support all features of OPC UA that are of interest for the product. OPC UA will allow systems to make product features available in a standard way, which they can not expose today without the new options provided by OPC UA.

For end users, it is important to have the wrappers and proxies available to migrate the large installed base of Classic OPC products to OPC UA. End users can install wrappers and proxies for tunneling Classic OPC through firewalls, including secure transmission over the internet and authenticated access, so they simply add value to existing industry proven solutions.

These components are only a first step. For vendors it is much more important that OPC UA offers abstract, modular, and simple base concepts in all areas of the standard, allowing an easy migration of existing OPC functionality to OPC UA. The very powerful concepts for extending this base enable vendors to expose more and more of their system features via OPC UA. This leads to an iterative development and improvement process.

## 1.8 Summary

### *1.8.1 Key Messages*

The OPC Foundation provides standards for data exchange in industrial automation. This includes the very successful OPC DA specification for current data, as well as OPC A&E for alarms and events and OPC HDA for historical data. All those specifications are based on the retiring COM/DCOM technology. A first step moving forward to state-of-the-art technologies was OPC XML-DA, which used only XML as data transport format and therefore did not meet the performance requirements of typical OPC applications.

With the lessons learned from OPC XML-DA the OPC Foundation created a new standard called Unified Architecture. Here, the transport can be done either using firewall-friendly Web Services by standards like SOAP and HTTP or an optimized binary TCP protocol for high performance communication. OPC UA provides interoperable and platform-independent, high-performing, scalable, secure, and reliable communication between applications. The technology switch from Microsoft's COM/DCOM to state-of-the-art and platform-independent transport protocols allows OPC UA applications to run on intelligent devices and controllers as well as in DCS and SCADA systems and up to the enterprise level with MES and ERP systems. This immensely increases the range of use compared to Classic OPC application

Besides the transport, the second big achievement of OPC UA is information modeling. OPC UA unifies the functionality of the different Classic OPC specifications by exposing current data, event notifications, and the history of both in one address space. It additionally provides a rich and extensible information model using object-oriented concepts, allowing meta data as well as complex data of your applications to be exposed. The extensible mechanisms allow defining standard information models by other organizations that can make use of the OPC UA communication infrastructure and focus on standardizing the information to be exposed. There are already initiatives going on defining standard information models, for example, FDI for device descriptions by mapping EDDL and FDT to OPC UA or for PLC programming languages by PLCopen. Like the transport capabilities the modeling capabilities of OPC UA scale well. You can keep the offered information simple, similar to Classic OPC, but you can also enrich the information with a type system and thus providing more information useful in many application scenarios.

### *1.8.2 Where to Find More Information?*

Information about Classic OPC can be found at the OPC Foundation web site (www.opcfoundation.org). There is also a book on Classic OPC [IL06]. Several vendors provide Classic OPC products and toolkits. A list can be found at www.opcconnect.com; here you will find overview information about OPC, including OPC UA. Additional links and information can be found on the book website www.opcuabook.com.

General information about OPC UA can be found at the OPC Foundation web site, including a special section on OPC UA (www.opcfoundation.org/ua). The web site also offers access to the OPC UA specifications where [UA Part 4] and [UA Part 6] focus on the data transport and [UA Part 3] and [UA Part 5] focus on modeling information.

### *1.8.3 What's Next?*

In the next three chapters, we will focus on how to provide information using OPC UA before we go into details on how to access those data (starting with Chap. 5).

In Chaps. 2 and 3, we introduce the base modeling concepts and give examples and best practice for these concepts. Standard information models are described in Chap. 4. This includes an introduction on how to create models: the description of the OPC defined models and an overview of standard information models defined by other organizations.

In Chap. 5 we will introduce the abstract Services of OPC UA, and Chap. 6 describes the mapping of those Services to concrete technologies like Web Services.